

Lógica digital



Objetivos de aprendizagem

Após ler este capítulo, você será capaz de:

- Compreender as operações básicas da álgebra booleana.
- Distinguir os diferentes tipos de flip-flops.
- Usar um mapa de Karnaugh a fim de simplificar uma expressão booleana.
- Apresentar uma visão geral dos dispositivos lógicos programáveis.

Álgebra booleana

- O circuito digital em computadores digitais e outros sistemas digitais é projetado, e seu comportamento é analisado, com o uso de uma disciplina matemática chamada de **álgebra booleana**.
- A álgebra booleana mostra ser uma ferramenta conveniente em duas áreas:
 1. **Análise:** é um modo econômico de descrever a função do circuito digital.
 2. **Projeto:** dada uma função desejada, a álgebra booleana pode ser aplicada a desenvolver uma implementação simplificada dessa função.

Álgebra booleana

- As operações de lógica básica são AND, OR e NOT, que são simbolicamente representadas por ponto, sinal de adição e barra sobrescrita:

$$A \text{ AND } B = A \cdot B$$

$$A \text{ OR } B = A + B$$

$$\text{NOT } A = \bar{A}$$

- Por exemplo, considere a equação

$$D = A + (\bar{B} \cdot C)$$

- D é igual a 1 se A for 1 ou se B = 0 e C = 1.
- Caso contrário, D = 0.

Álgebra booleana

- Operadores booleanos de duas variáveis de entrada:

P	Q	NOT P ($\overline{P}$)	P AND Q ($P \cdot Q$)	P OR Q ($P + Q$)	P NAND Q ($\overline{P \cdot Q}$)	P NOR Q ($\overline{P + Q}$)	P XOR Q ($P \oplus Q$)
0	0	1	0	0	1	1	0
0	1	1	0	1	1	0	1
1	0	0	0	1	1	0	1
1	1	0	1	1	0	0	0

Álgebra booleana

- Operadores booleanos estendidos para mais de duas entradas ($A, B, \dots$):

Operação	Expressão	Saída = 1 if
AND	$A \cdot B \cdot \dots$	Todos do conjunto $\{A, B, \dots\}$ são 1.
OR	$A + B + \dots$	Qualquer um do conjunto $\{A, B, \dots\}$ é 1.
NAND	$\overline{A \cdot B \cdot \dots}$	Qualquer um do conjunto $\{A, B, \dots\}$ é 0.
NOR	$\overline{A + B + \dots}$	Todos do conjunto $\{A, B, \dots\}$ são 0.
XOR	$A \oplus B \oplus \dots$	O conjunto $\{A, B, \dots\}$ contém um número ímpar de uns.

Álgebra booleana

➤ Identidades básicas da álgebra booleana:

Postulados básicos

$A \cdot B = B \cdot A$	$A + B = B + A$	Propriedade comutativa
$A \cdot (B + C) = (A \cdot B) + (A \cdot C)$	$A + (B \cdot C) = (A + B) \cdot (A + C)$	Propriedade distributiva
$1 \cdot A = A$	$0 + A = A$	Elementos de identidade
$A \cdot \overline{A} = 0$	$A + \overline{A} = 1$	Elementos de inversão

Outras identidades

$0 \cdot A = 0$	$1 + A = 1$	
$A \cdot A = A$	$A + A = A$	
$A \cdot (B \cdot C) = (A \cdot B) \cdot C$	$A + (B + C) = (A + B) + C$	Propriedade associativa
$\overline{A \cdot B} = \overline{A} + \overline{B}$	$\overline{A + B} = \overline{A} \cdot \overline{B}$	Teorema de DeMorgan

Portas lógicas

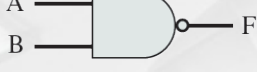
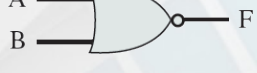
- Uma **porta lógica** é um circuito eletrônico que produz um sinal de saída que é uma operação booleana simples em seus sinais de entrada.
- As portas lógicas básicas usadas na lógica digital são:
 1. AND,
 2. OR,
 3. NOT,
 4. NAND,
 5. NOR e
 6. XOR.

Portas lógicas

WILLIAM STALLINGS

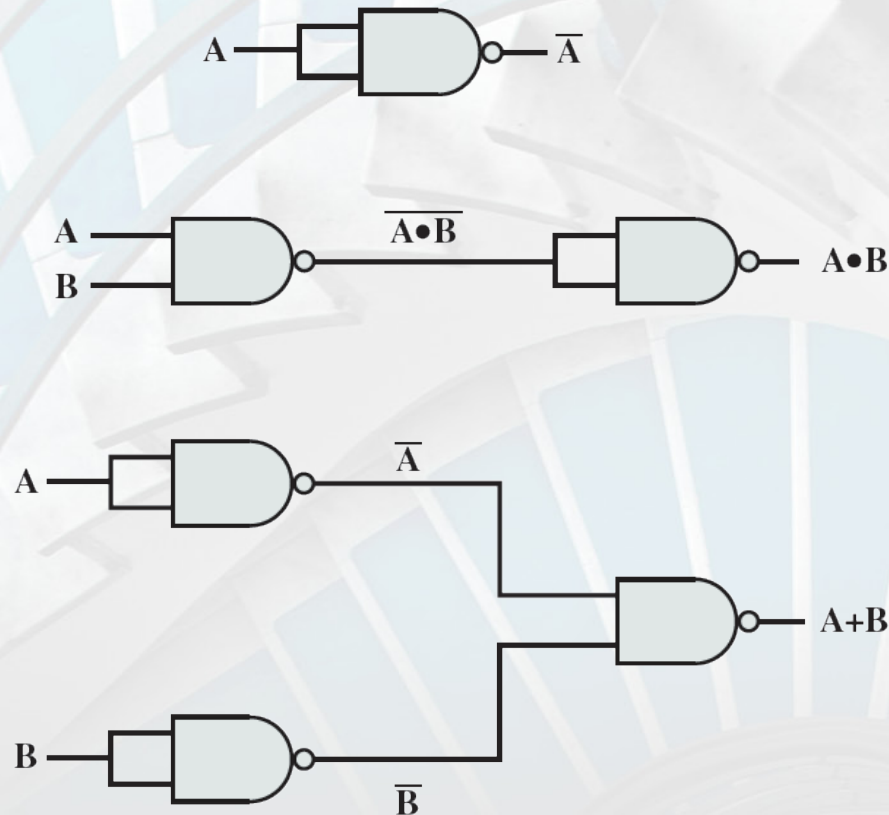
Arquitetura e Organização de Computadores

10ª edição

Nome	Símbolo gráfico	Função algébrica	Tabela verdade															
AND		$F = A \cdot B$ ou $F = AB$	<table><tr><th>A</th><th>B</th><th>F</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	A	B	F	0	0	0	0	1	0	1	0	0	1	1	1
A	B	F																
0	0	0																
0	1	0																
1	0	0																
1	1	1																
OR		$F = A + B$	<table><tr><th>A</th><th>B</th><th>F</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	A	B	F	0	0	0	0	1	1	1	0	1	1	1	1
A	B	F																
0	0	0																
0	1	1																
1	0	1																
1	1	1																
NOT		$F = \overline{A}$ ou $F = A'$	<table><tr><th>A</th><th>F</th></tr><tr><td>0</td><td>1</td></tr><tr><td>1</td><td>0</td></tr></table>	A	F	0	1	1	0									
A	F																	
0	1																	
1	0																	
NAND		$F = \overline{AB}$	<table><tr><th>A</th><th>B</th><th>F</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	A	B	F	0	0	1	0	1	1	1	0	1	1	1	0
A	B	F																
0	0	1																
0	1	1																
1	0	1																
1	1	0																
NOR		$F = \overline{A + B}$	<table><tr><th>A</th><th>B</th><th>F</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	A	B	F	0	0	1	0	1	0	1	0	0	1	1	0
A	B	F																
0	0	1																
0	1	0																
1	0	0																
1	1	0																
XOR		$F = A \oplus B$	<table><tr><th>A</th><th>B</th><th>F</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	A	B	F	0	0	0	0	1	1	1	0	1	1	1	0
A	B	F																
0	0	0																
0	1	1																
1	0	1																
1	1	0																

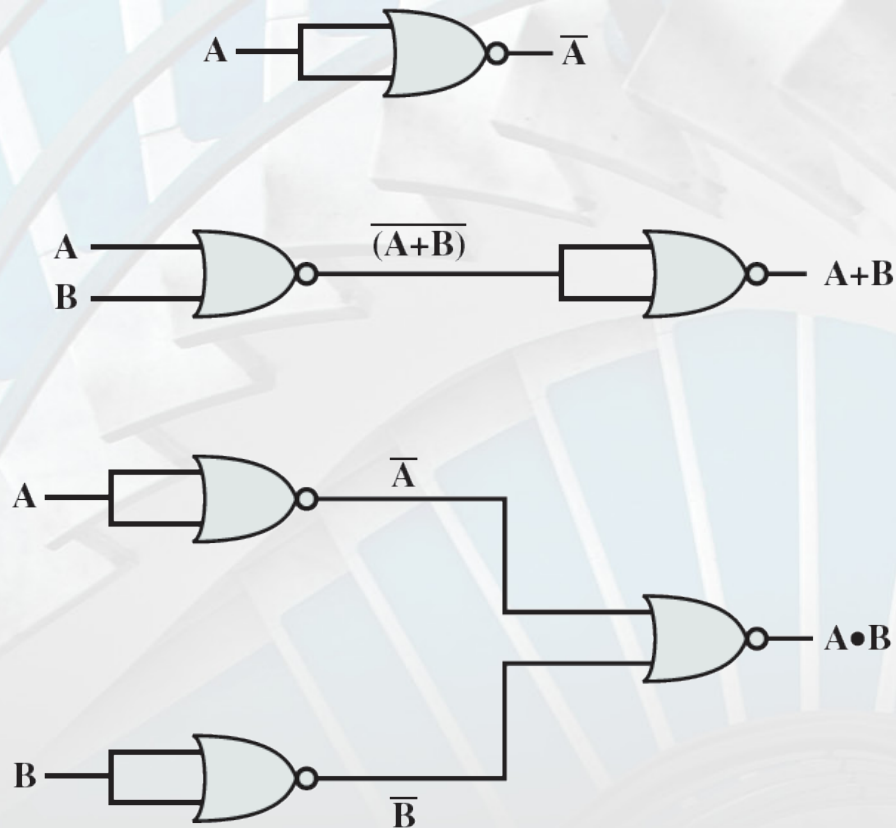
Portas lógicas

- Alguns usos de portas NAND:



Portas lógicas

- Alguns usos de portas NOR:



Circuitos combinacionais

- Um **circuito combinacional** é um conjunto de portas lógicas interconectadas cuja saída em qualquer tempo é uma função somente da entrada nesse tempo.
- Em termos gerais, um circuito combinacional consiste em n entradas binárias e m saídas binárias.
- Como a porta, um circuito combinacional pode ser definido de três maneiras:
 1. Tabela verdade
 2. Símbolos gráficos
 3. Equações booleanas

Implementação das funções booleanas

WILLIAM STALLINGS

**Arquitetura e
Organização de
Computadores**

10ª edição

- Uma função booleana de três variáveis:

A	B	C	F
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	0

Implementação das funções booleanas

WILLIAM STALLINGS
**Arquitetura e
Organização de
Computadores**

10ª edição

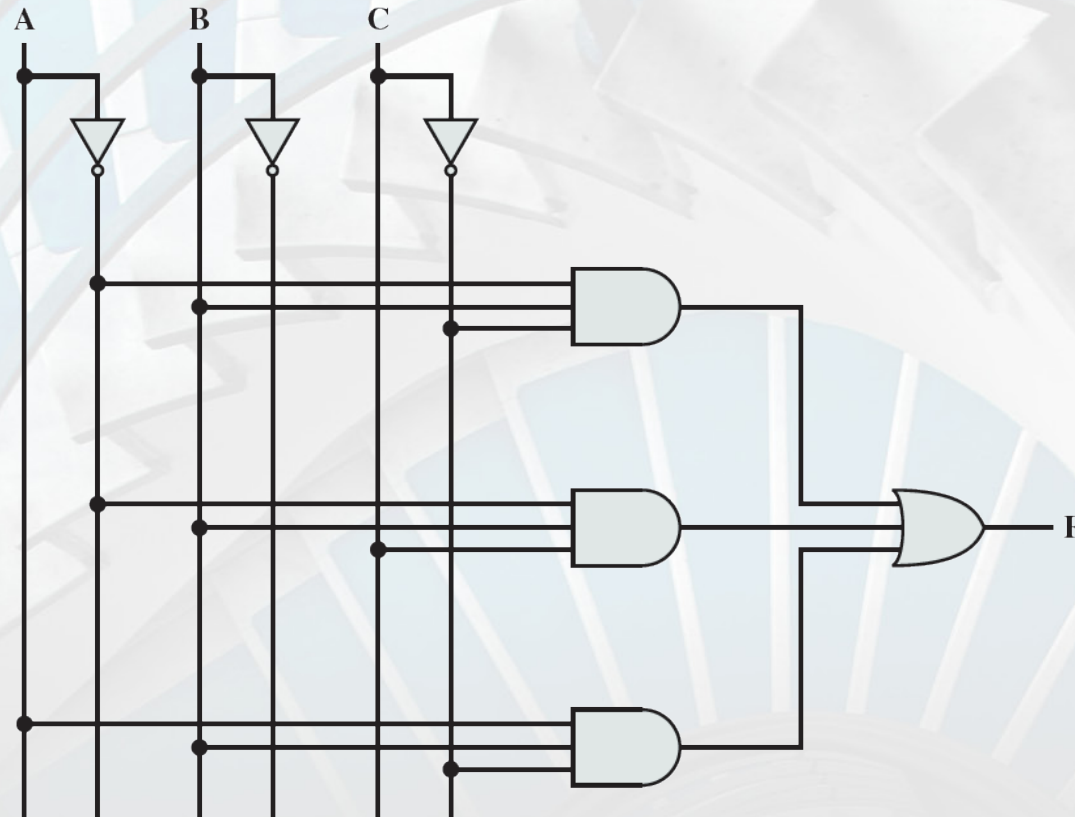
- Considere a função booleana representada pela tabela verdade na tabela anterior.
- Podemos expressar essa função simplesmente discriminando as combinações dos valores de A, B e C, que fazem com que F seja 1:
$$F = \overline{A}B\overline{C} + \overline{A}BC + A\overline{B}\overline{C}$$
- Existem três combinações de valores de entrada que fazem com que F seja 1, e se alguma dessas combinações ocorrer, o resultado é 1.
- Essa forma de expressão, por razões evidentes, é conhecida como **soma de produtos** (**SOP** – do inglês *Sum Of Products*).

Implementação das funções booleanas

WILLIAM STALLINGS
**Arquitetura e
Organização de
Computadores**

10ª edição

- Implementação da soma de produtos da tabela anterior:



Implementação das funções booleanas

WILLIAM STALLINGS
**Arquitetura e
Organização de
Computadores**

10ª edição

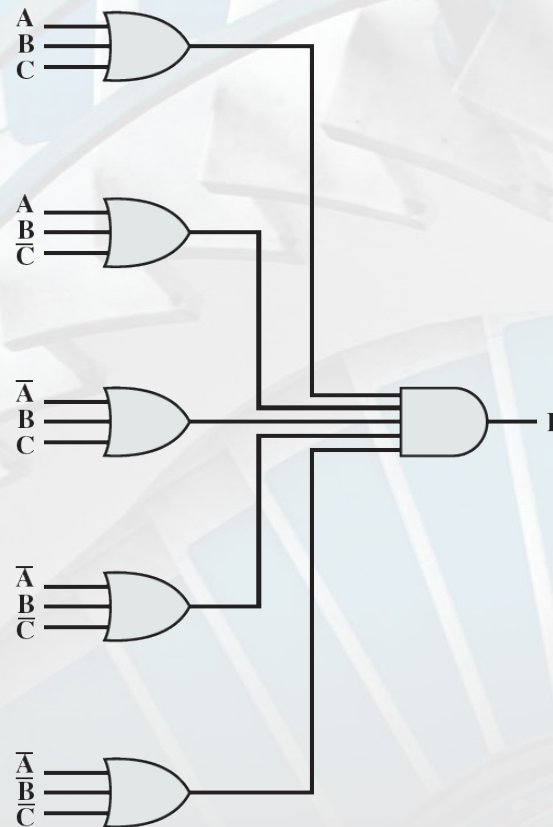
- Podemos também dizer que a saída é 1 se nenhuma das combinações de entrada que produzem 0 for verdadeira.
- Desse modo, $F = \overline{(\overline{A} \overline{B} \overline{C})} \cdot \overline{(\overline{A} \overline{B} C)} \cdot \overline{(A \overline{B} \overline{C})} \cdot \overline{(A \overline{B} C)} \cdot \overline{(A B C)}$
- Isso pode ser reescrito usando a generalização do teorema de DeMorgan: $\overline{(X \cdot Y \cdot Z)} = \overline{X} + \overline{Y} + \overline{Z}$
- Desse modo, $F = (\overline{\overline{A}} + \overline{\overline{B}} + \overline{\overline{C}}) \cdot (\overline{\overline{A}} + \overline{\overline{B}} + \overline{\overline{C}}) \cdot (\overline{\overline{A}} + \overline{\overline{B}} + \overline{\overline{C}}) \cdot (\overline{\overline{A}} + \overline{\overline{B}} + \overline{\overline{C}}) \cdot (\overline{\overline{A}} + \overline{\overline{B}} + \overline{\overline{C}})$
 $= (A + B + C) \cdot (A + B + \overline{C}) \cdot (\overline{A} + B + C) \cdot (\overline{A} + B + \overline{C}) \cdot (\overline{A} + \overline{B} + \overline{C})$
- Está na forma do **produto de somas (POS** – do inglês, *Product Of Sums*).

Implementação das funções booleanas

WILLIAM STALLINGS
**Arquitetura e
Organização de
Computadores**

10ª edição

- Implementação do produto de somas da tabela anterior:

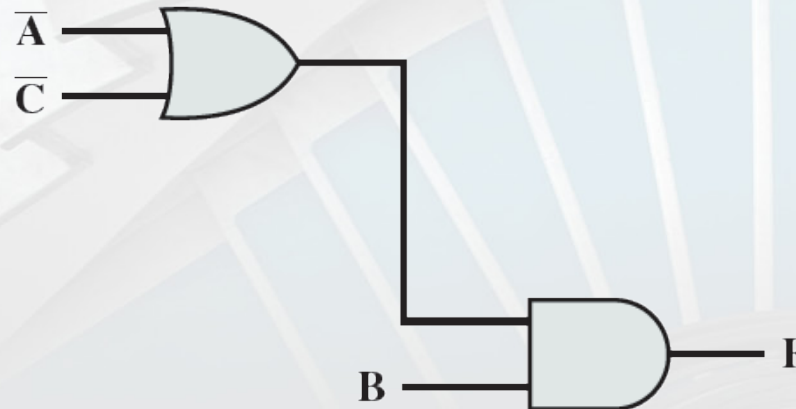


Implementação das funções booleanas

WILLIAM STALLINGS
**Arquitetura e
Organização de
Computadores**

10ª edição

- Para a **simplificação algébrica**, algum pensamento deve convencer o leitor de que uma expressão equivalente é $F = \overline{A}B + B\overline{C}$
- Ou, mesmo mais simples, $F = B(\overline{A} + \overline{C})$



Implementação das funções booleanas

WILLIAM STALLINGS
**Arquitetura e
Organização de
Computadores**

10ª edição

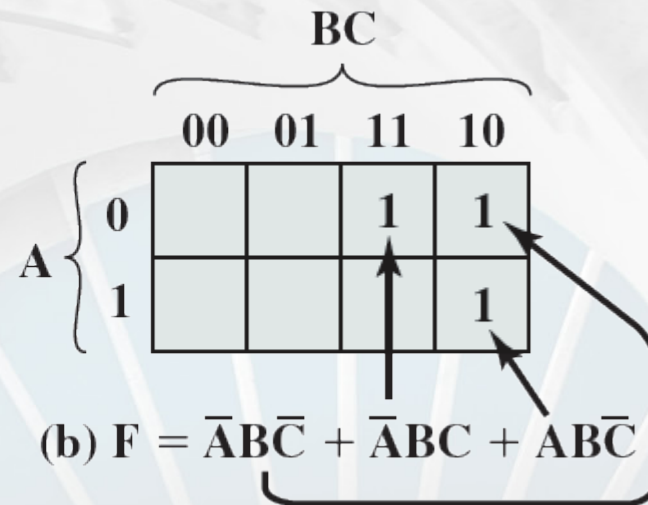
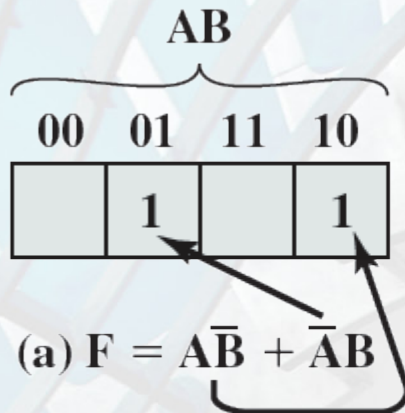
- Por propósito de simplificação, o **mapa de Karnaugh** é uma maneira conveniente de representar uma função booleana de um pequeno número (até quatro) de variáveis.
- O mapa é uma matriz de 2^n quadrados, representando todas as combinações possíveis de valores de n variáveis binárias.
- Cada quadrado corresponde a um único produto na forma de soma dos produtos, com um valor 1 correspondendo à variável e um valor 0 correspondendo ao NOT dessa variável.
- Para cada produto na função, 1 é colocado no quadrado correspondente.

Implementação das funções booleanas

WILLIAM STALLINGS
**Arquitetura e
Organização de
Computadores**

10ª edição

- Uso dos mapas de Karnaugh com o objetivo de representar as funções booleanas:

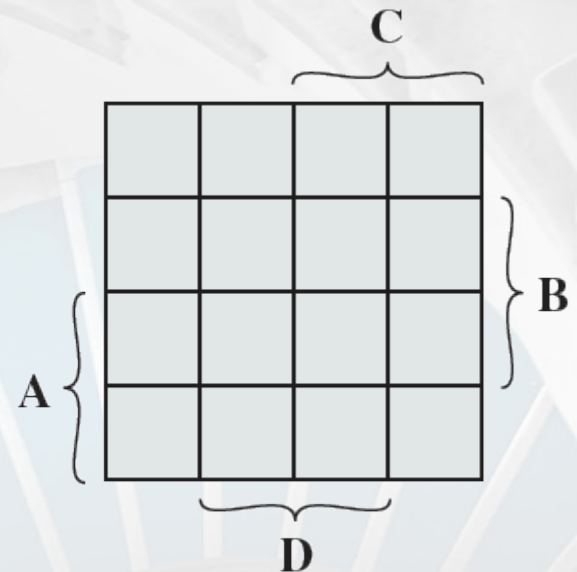
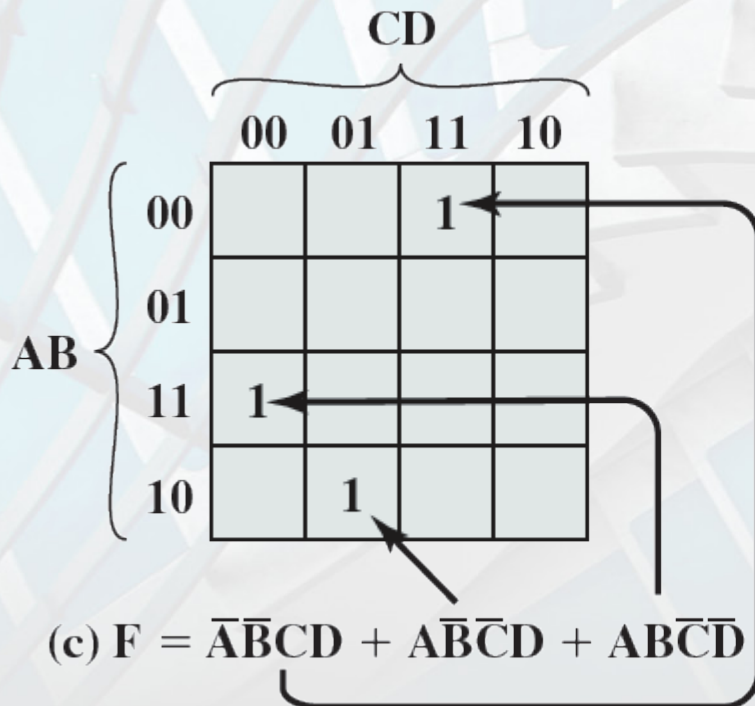


Implementação das funções booleanas

WILLIAM STALLINGS
**Arquitetura e
Organização de
Computadores**

10ª edição

- Uso dos mapas de Karnaugh com o objetivo de representar as funções booleanas



(d) Definição simplificada do mapa

Implementação das funções booleanas

WILLIAM STALLINGS
Arquitetura e
Organização de
Computadores

10ª edição

Podemos resumir as regras para simplificação da seguinte maneira:

1. Entre os quadrados marcados, encontre aqueles que pertencem a um único bloco maior que 1, 2, 4 ou 8 e circule esses blocos.
2. Selecione blocos adicionais de quadrados marcados que são tão grandes quanto possível e tão poucos quanto possível, mas inclua todos os blocos marcados pelo menos uma vez.
3. Continue a desenhar alças ao redor dos quadrados marcados sozinhos, dos pares de quadrados marcados ou dos grupos de quatro, oito e assim por diante; então use o mínimo possível desses blocos para incluir todos os quadrados marcados.

Implementação das funções booleanas

WILLIAM STALLINGS
**Arquitetura e
Organização de
Computadores**

10ª edição

- Grupos de sobreposição:

		BC			
		00	01	11	10
A	0			1	1
	1				1

(a) $F = \bar{A}B + B\bar{C}$

Implementação das funções booleanas

WILLIAM STALLINGS
**Arquitetura e
Organização de
Computadores**

10ª edição

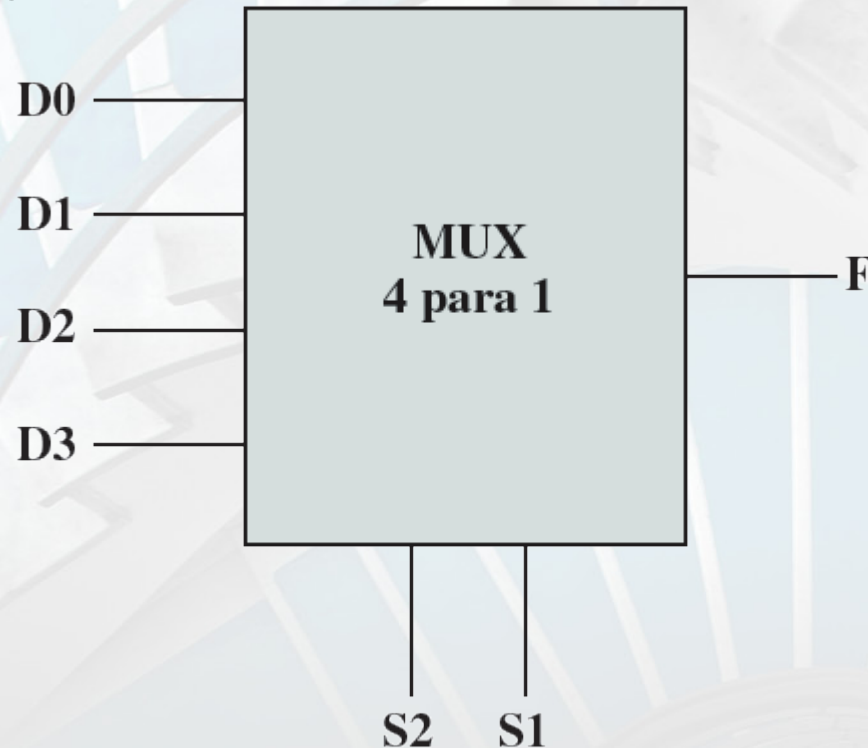
- Grupos de sobreposição:

		CD			
		00	01	11	10
AB	00				
	01		1		
	11		1	1	
	10			1	

(b) $F = \overline{B}CD + ACD$

Multiplexadores

- O **multiplexador** conecta diversas entradas a uma única saída:



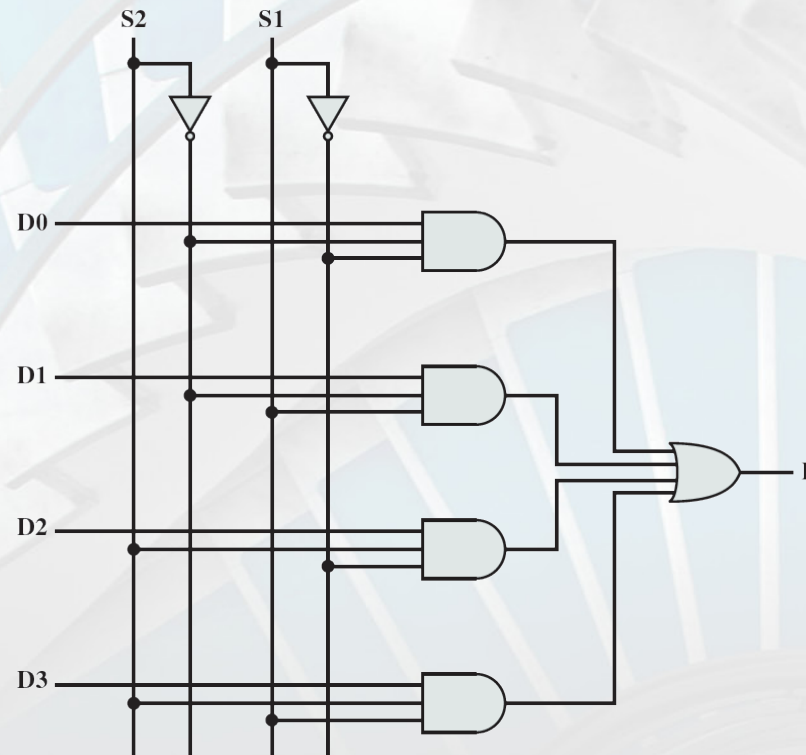
Multiplexadores

- Tabela verdade do multiplexador 4 para 1:

S2	S1	F
0	0	D0
0	1	D1
1	0	D2
1	1	D3

Multiplexadores

- A figura abaixo apresenta uma implementação com o uso de portas AND, OR e NOT:

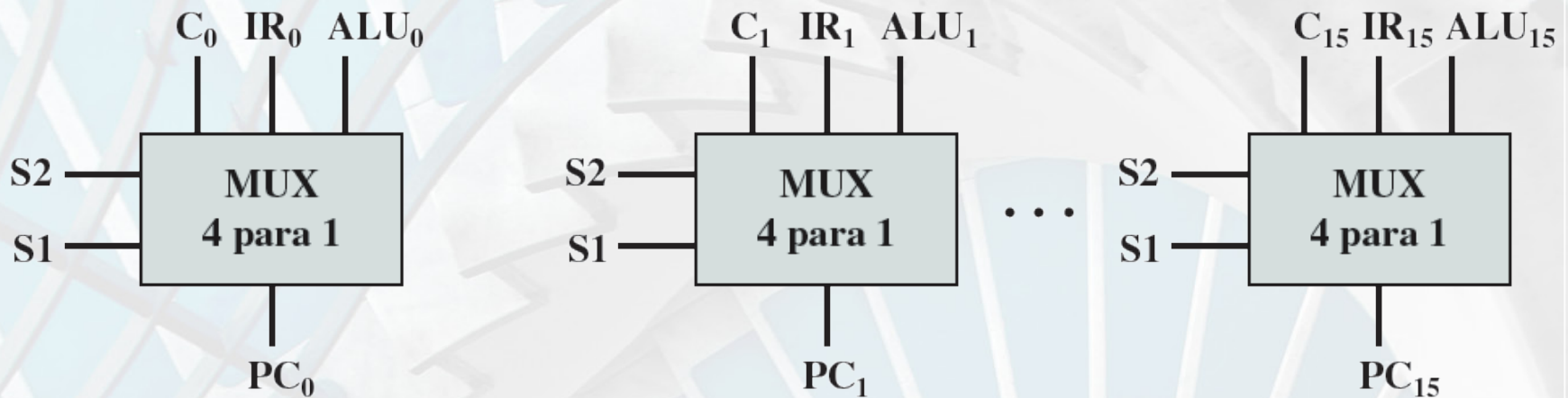


Multiplexadores

- Os multiplexadores são usados em circuitos digitais para controlar sinais e rotear dados.
- Um exemplo é o carregamento de um **contador de programa (PC)**, do inglês *Program Counter*).
- O valor a ser carregado no contador de programa pode vir de uma de várias fontes distintas:
 1. Um contador binário
 2. O registrador de instruções
 3. A saída da ALU

Multiplexadores

- Entrada de multiplexador para o contador de programa:

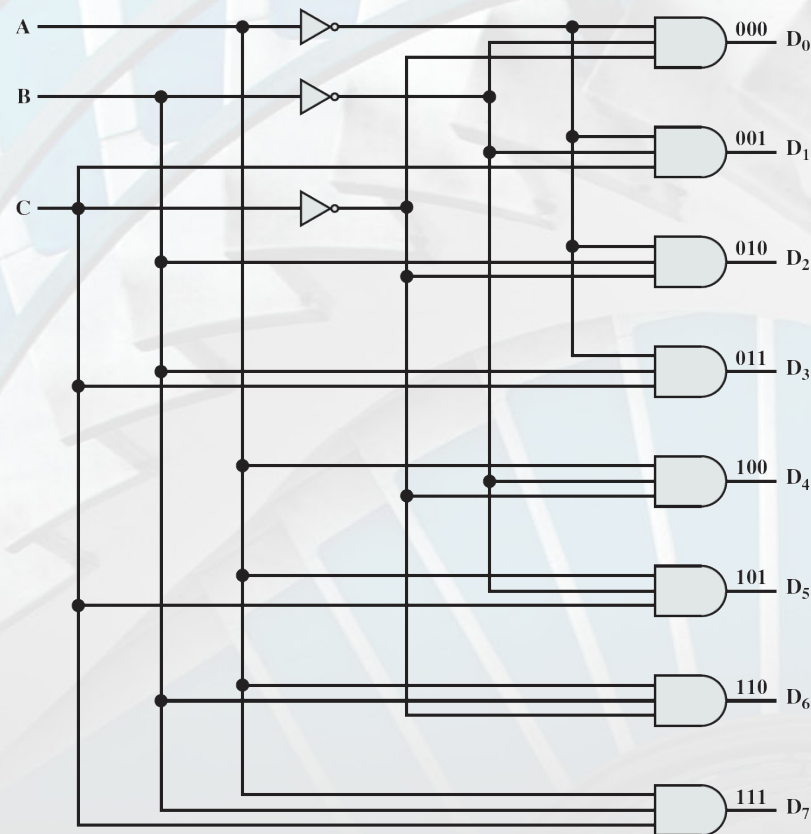


Decodificadores

- Um **decodificador** é um circuito combinacional com um número de linhas de saída, em que somente uma delas é acionada por vez.
- Qual linha de saída será acionada vai depender do padrão das linhas de entrada.
- Em geral, um decodificador tem n entradas e 2^n saídas.
- A figura a seguir apresenta um decodificador com três entradas e oito saídas.

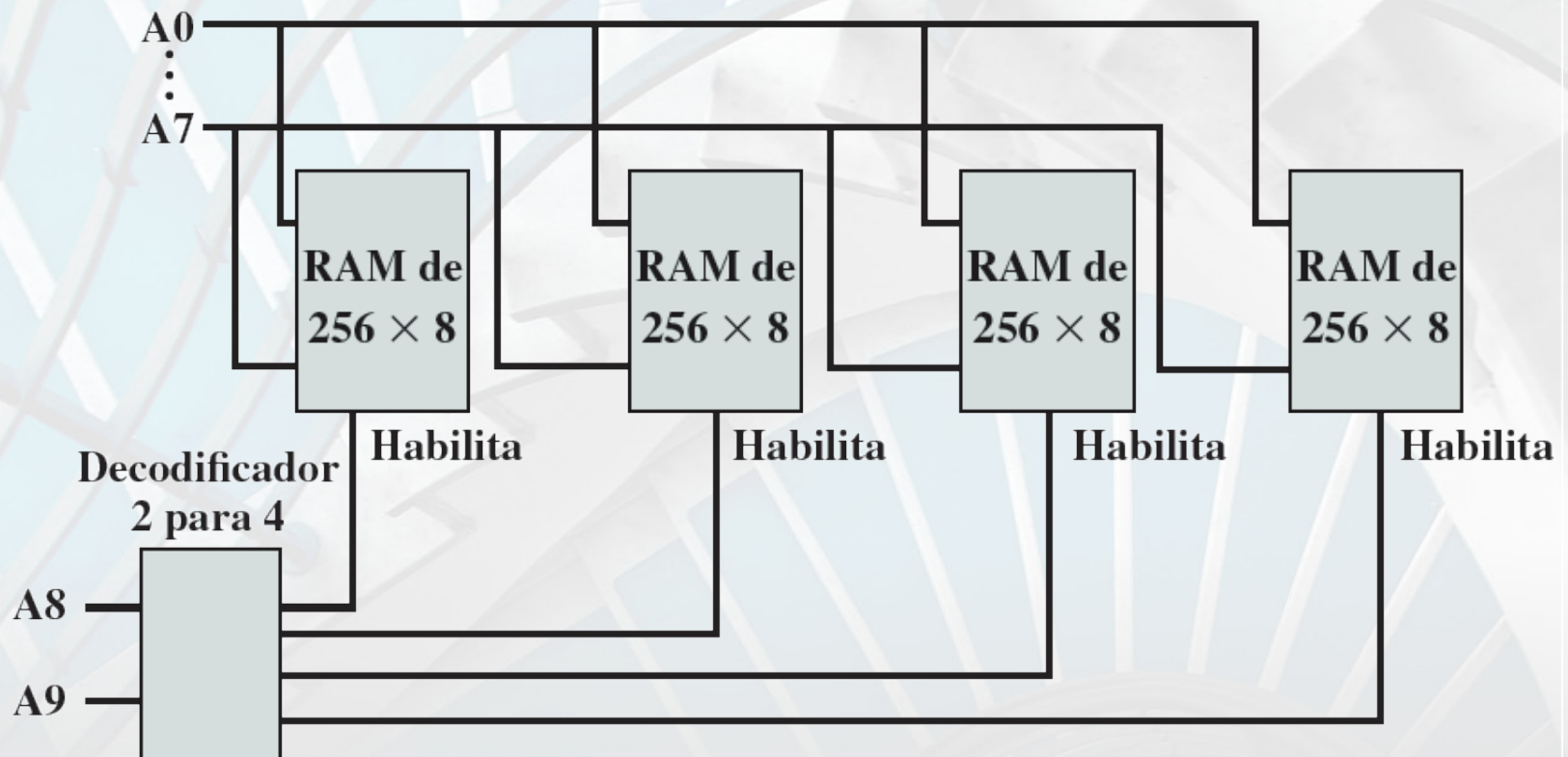
Decodificadores

- Decodificador com 3 entradas e $2^3 = 8$ saídas:



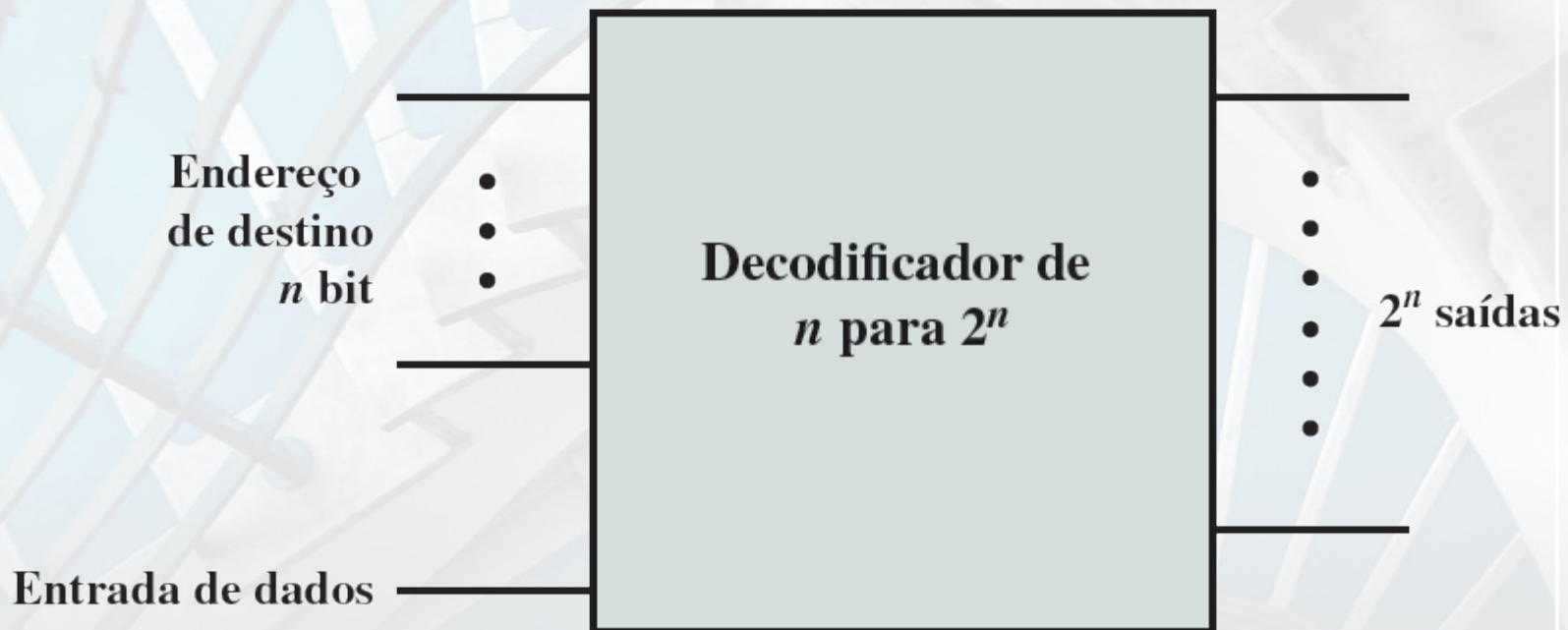
Decodificadores

- Decodificação de endereço:



Decodificadores

- Implementação de um demultiplexador usando um decodificador:



Memória somente de leitura

- Existe um tipo de memória que é implementado com circuitos combinacionais, chamado de **memória somente de leitura (ROM** – do inglês, *Read Only Memory*).
- Lembre-se de que uma ROM é uma unidade de memória que desempenha somente operação de leitura.
- Isso implica que a informação binária armazenada em uma ROM é permanente e foi criada durante o processo de fabricação.
- Desse modo, uma dada entrada à ROM (linhas de endereço) sempre produz a mesma saída (linhas de dados).

Memória somente de leitura

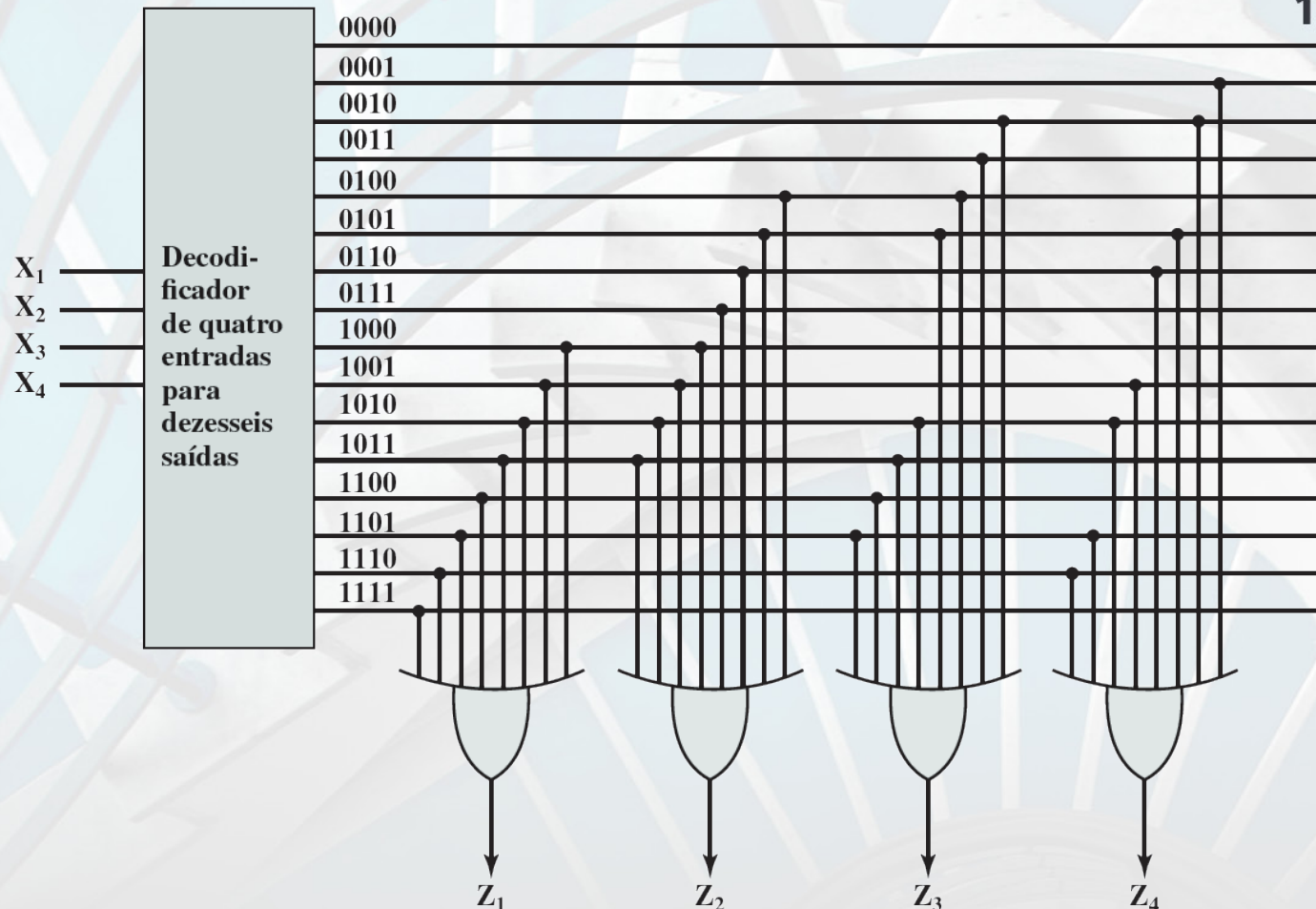
- Como as saídas são uma função somente das entradas presentes, a ROM é na verdade um circuito combinacional.
- Uma ROM pode ser implementada com um decodificador e um conjunto de portas OR.
- A figura a seguir mostra como essa memória pode ser implementada usando um decodificador de 4 para 16 e quatro portas OR.
- Como acontece com um array lógico programável, uma organização regular é usada, e as interconexões são feitas para refletir o resultado desejado.

Memória somente de leitura

WILLIAM STALLINGS

**Arquitetura e
Organização de
Computadores**

10ª edição



Somadores

WILLIAM STALLINGS

Arquitetura e Organização de Computadores

10ª edição

- Tabelas verdade de adição binária - Adição de um bit:

A	B	Soma	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Somadores

WILLIAM STALLINGS

**Arquitetura e
Organização de
Computadores**

10ª edição

- Tabelas verdade de adição binária - Adição com entrada de carry:

C_{in}	A	B	Soma	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Somadores

- A adição binária difere da álgebra booleana porque o resultado inclui um termo de *carry*.
- Desse modo,

0	0	1	1
$\frac{+0}{0}$	$\frac{+1}{1}$	$\frac{+0}{1}$	$\frac{+1}{10}$
- Para um somador de múltiplos bits funcionar, cada um dos somadores de um bit deve ter três entradas, inclusive o *carry* a partir do próximo somador de menor ordem.
- Um somador de 4 bits é descrito na figura a seguir.

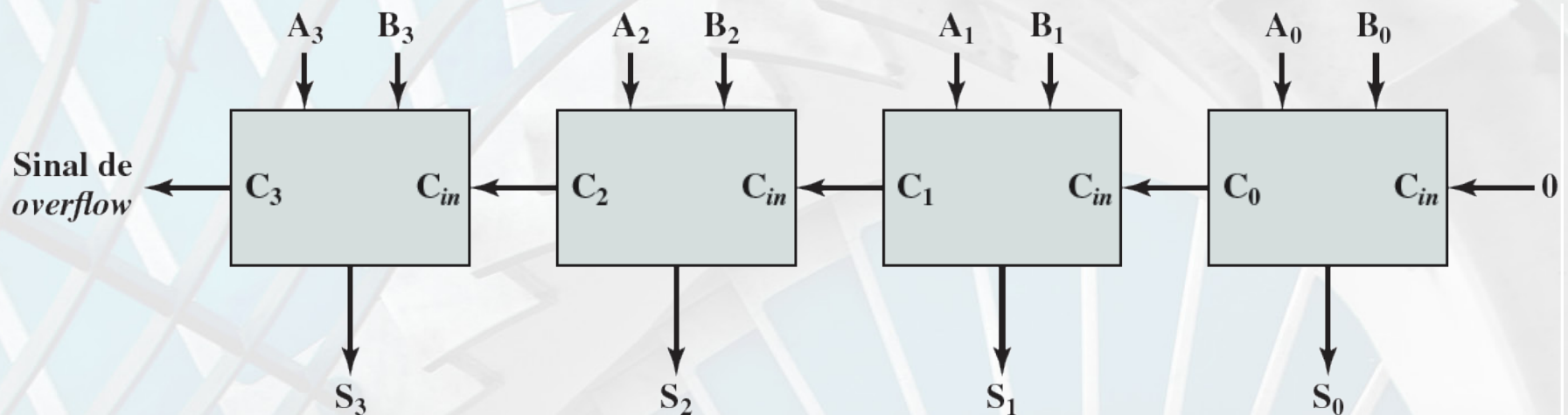
Somadores

WILLIAM STALLINGS

Arquitetura e
Organização de
Computadores

10ª edição

- Somador de 4 bits:



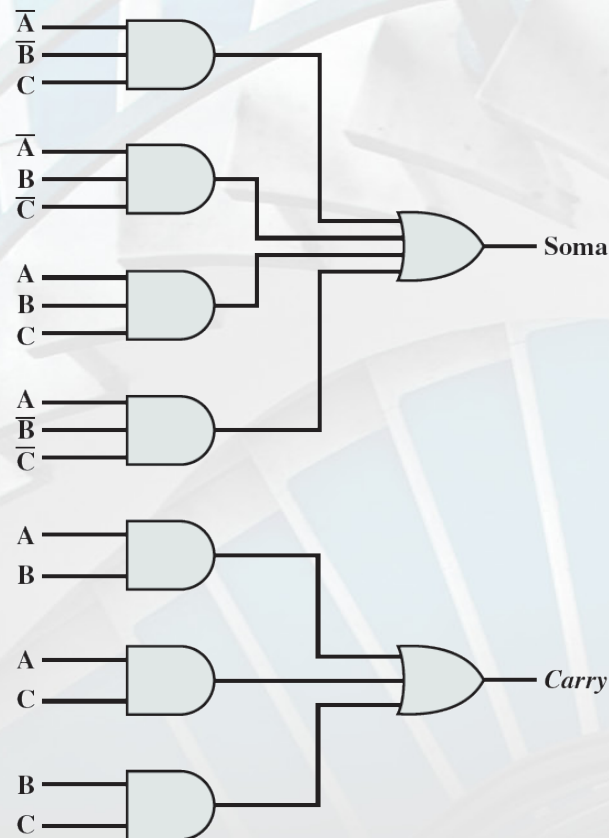
Somadores

WILLIAM STALLINGS

**Arquitetura e
Organização de
Computadores**

10ª edição

- Implementação de um somador:



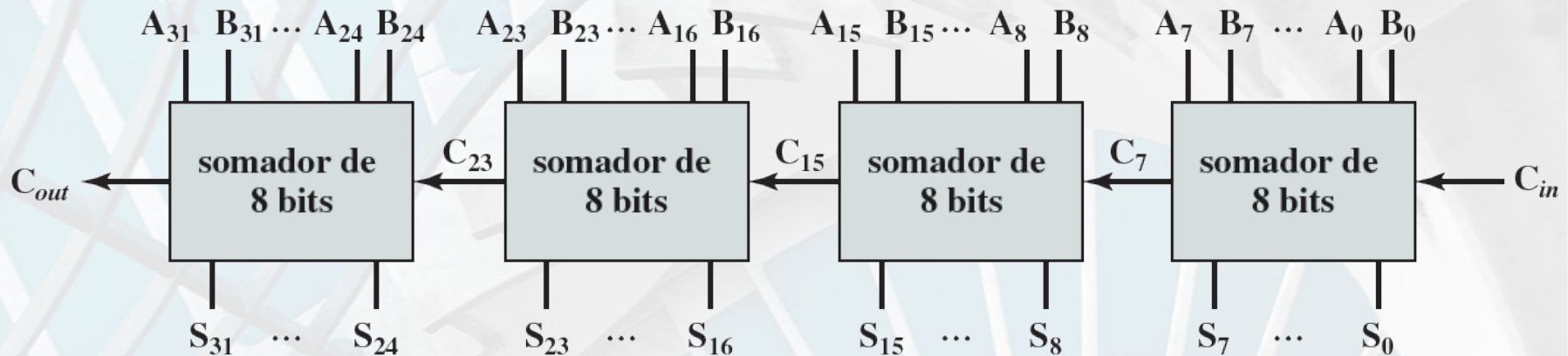
Somadores

WILLIAM STALLINGS

**Arquitetura e
Organização de
Computadores**

10ª edição

- Construção de um somador de 32 bits usando somadores de 8 bits:



Somadores

- Propomos uma expressão que especifique a entrada de *carry* para algum estágio do somador sem referência a valores prévios de *carry*.

$$C_0 = A_0B_0$$

- Temos:

$$C_1 = A_1B_1 + A_1A_0B_0 + B_1A_0B_0$$

- Seguindo esse procedimento, temos:

$$C_2 = A_2B_2 + A_2A_1B_1 + A_2A_1A_0B_0 + A_2B_1A_0B_0 + B_2A_1B_1 + B_2A_1A_0B_0 + B_2B_1A_0B_0$$

- Cada termo *carry* pode ser expresso na forma SOP como uma função somente de entradas originais, sem dependência de *carries*.

Circuitos sequenciais

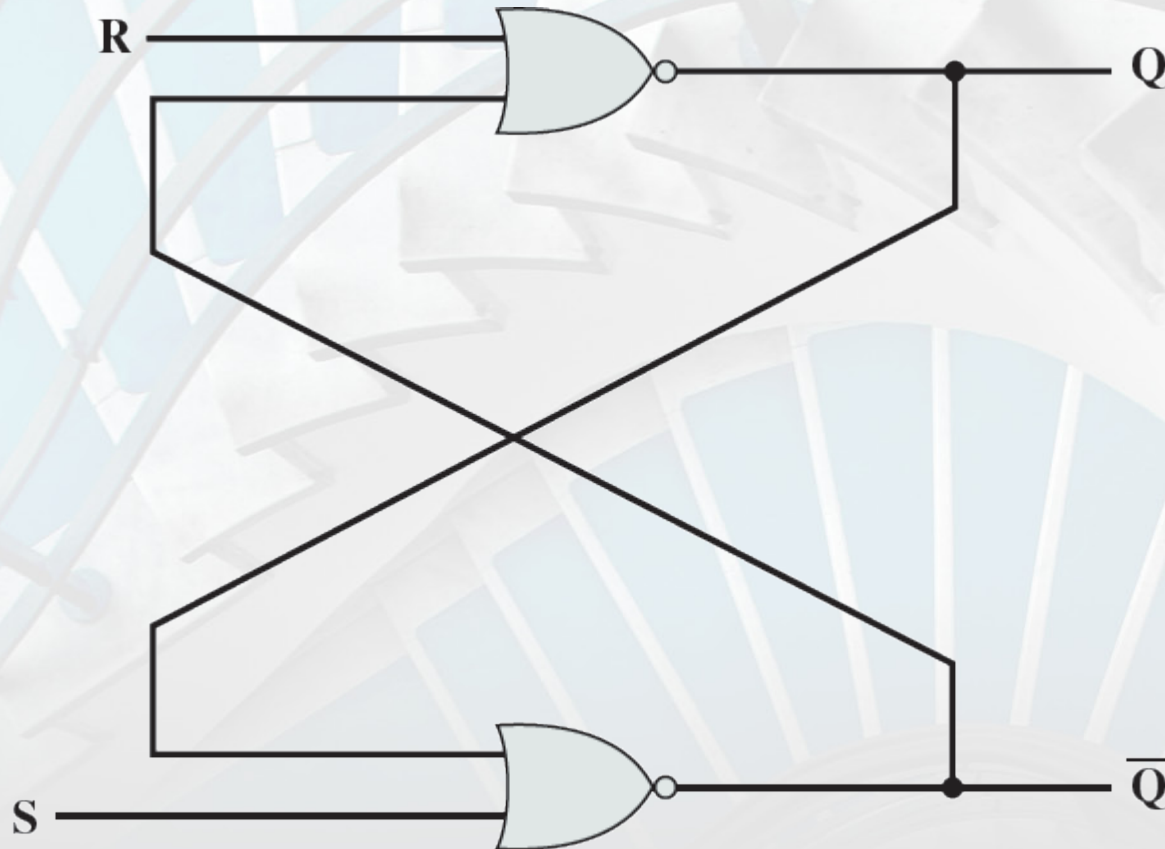
- Uma forma mais complexa de circuito lógico digital é o **circuito sequencial**.
- A saída atual de um circuito sequencial depende não somente da entrada atual, mas também do histórico das entradas.
- Outra forma, geralmente mais útil, de visualizar isso é que a saída atual do circuito sequencial depende da entrada e do estado atual desse circuito.
- A forma mais simples de circuito sequencial é o **flip-flop**.

Flip-flops

- Existe uma ampla gama de **flip-flops**, e todos eles compartilham duas propriedades:
 1. O flip-flop é um dispositivo biestável.
 2. O flip-flop tem duas saídas, que são sempre complementos uma da outra.
- A figura a seguir mostra uma configuração comum, conhecida como flip-flop S-R ou **latch S-R**.
- O circuito tem duas entradas e duas saídas, e consiste em duas portas NOR conectadas com realimentação.

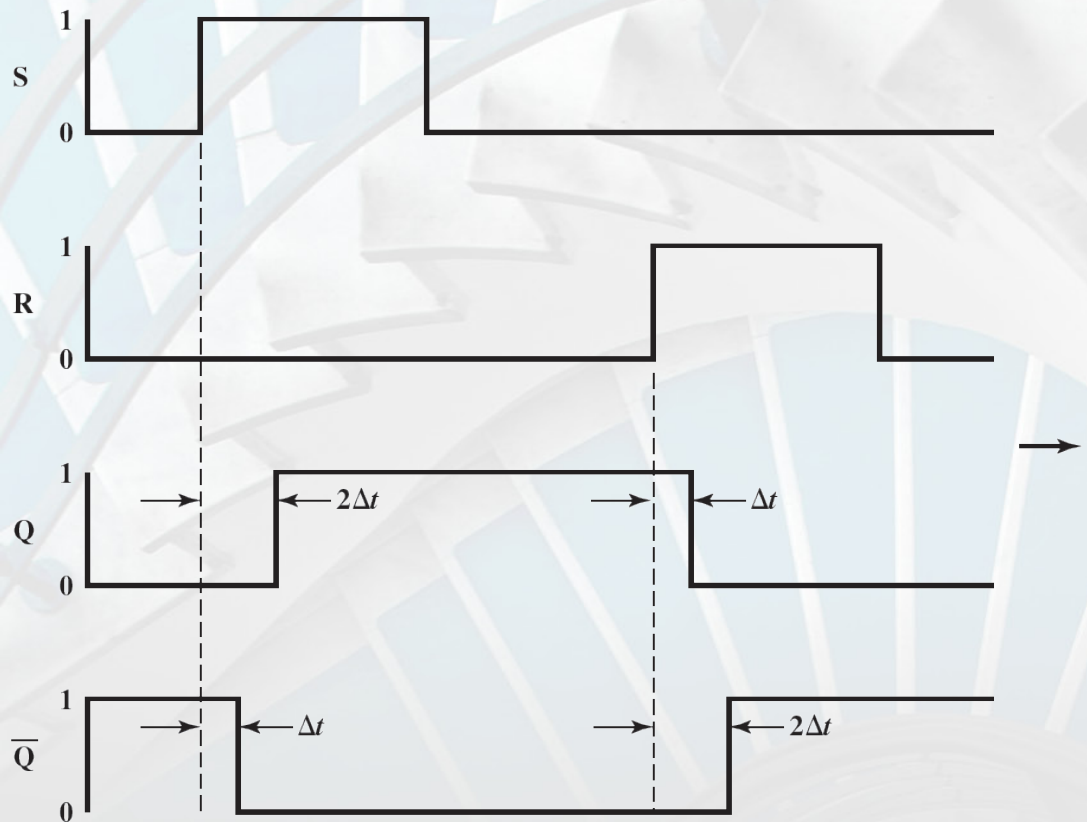
Flip-flops

- Latch S-R implementado com portas NOR:



Flip-flops

- Diagrama de temporização do latch S-R com NOR:

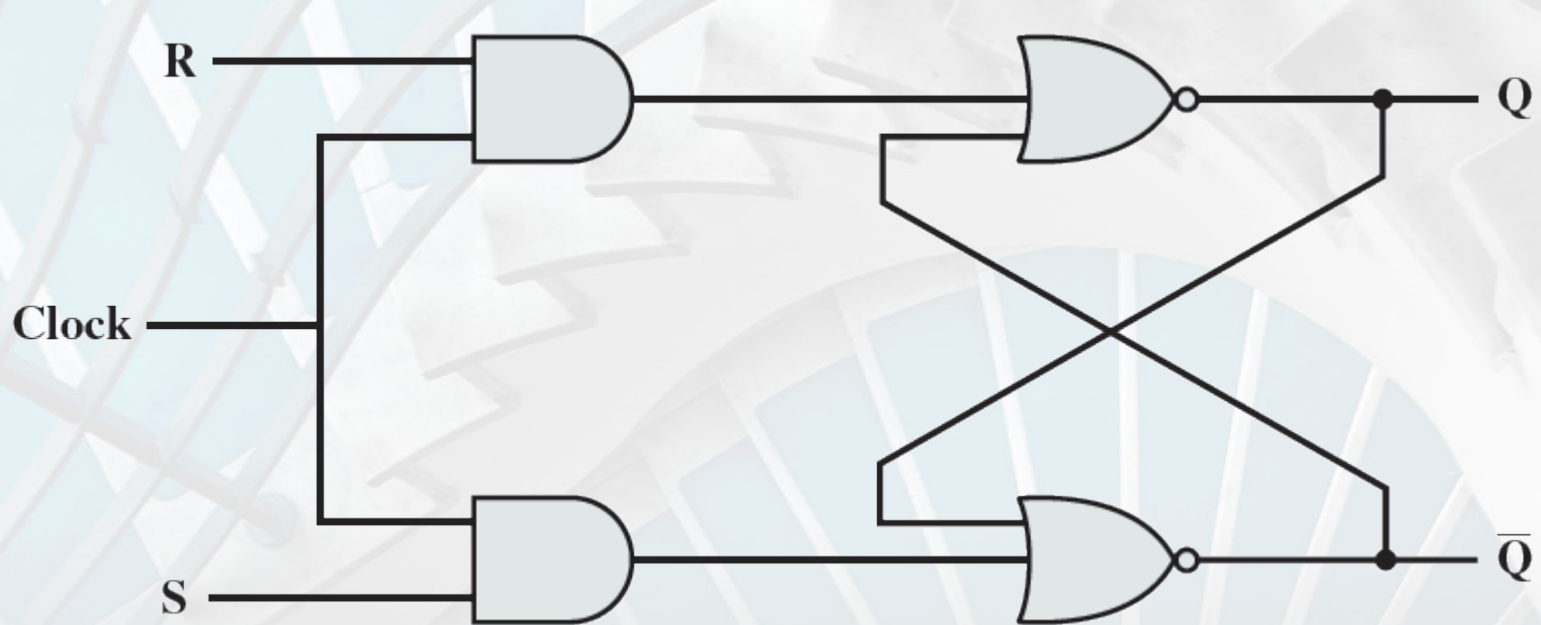


Flip-flops

- A saída do latch S-R muda, depois de um breve atraso de tempo, em resposta à alteração na entrada.
- Mais tipicamente, os eventos no computador digital são sincronizados por um pulso de clock, de modo que as mudanças ocorrem somente quando um pulso de clock acontece.
- A figura a seguir mostra essa disposição.
- Esse dispositivo é denominado **flip-flop S-R com clock**.

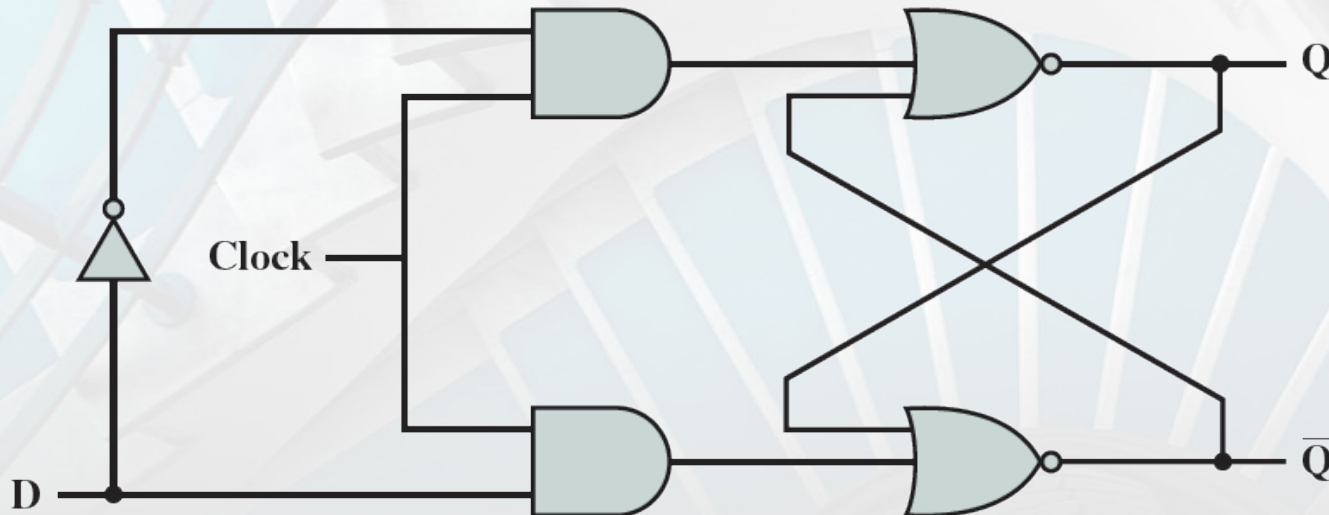
Flip-flops

- Flip-flop de S-R com clock:



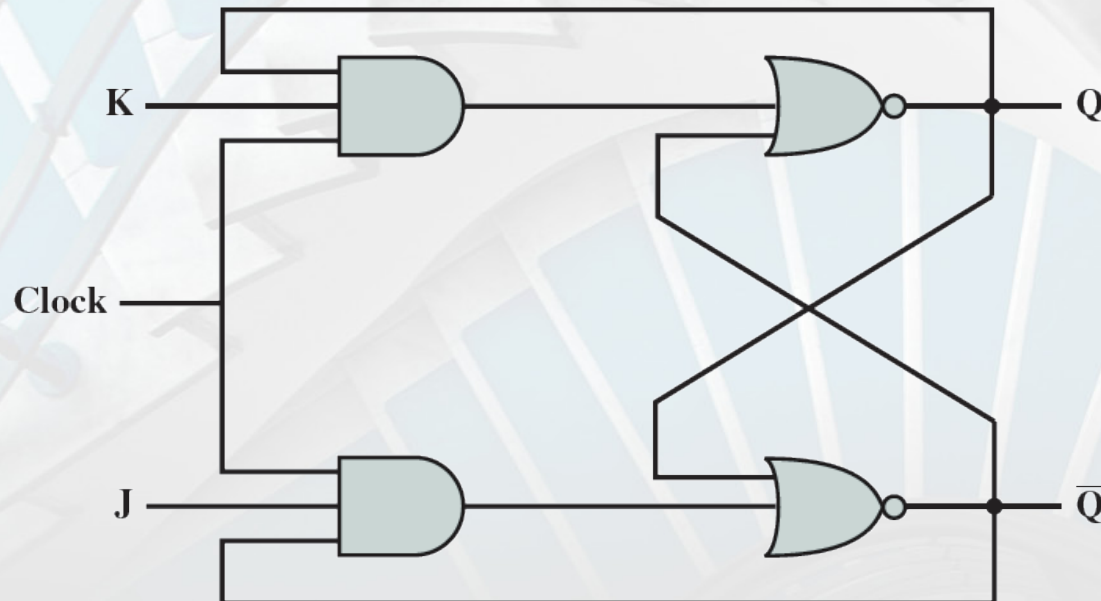
Flip-flops

- Um problema com o flip-flop S-R é que a condição $R = 1, S = 1$ deve ser evitada.
- Uma forma de fazer isso é permitir apenas uma única entrada. **O flip-flop D** realiza isso:



Flip-flops

- O flip-flop J-K tem duas entradas.
- Todavia, nesse caso todas as combinações possíveis nos valores de entrada são válidas:



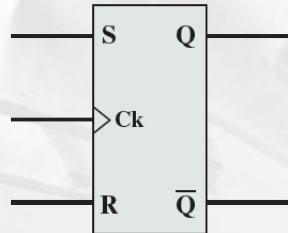
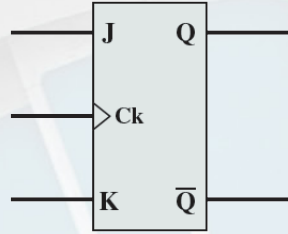
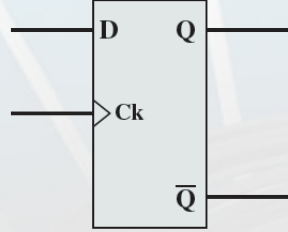
Flip-flops

WILLIAM STALLINGS

Arquitetura e Organização de Computadores

10ª edição

➤ Flip-flops básicos:

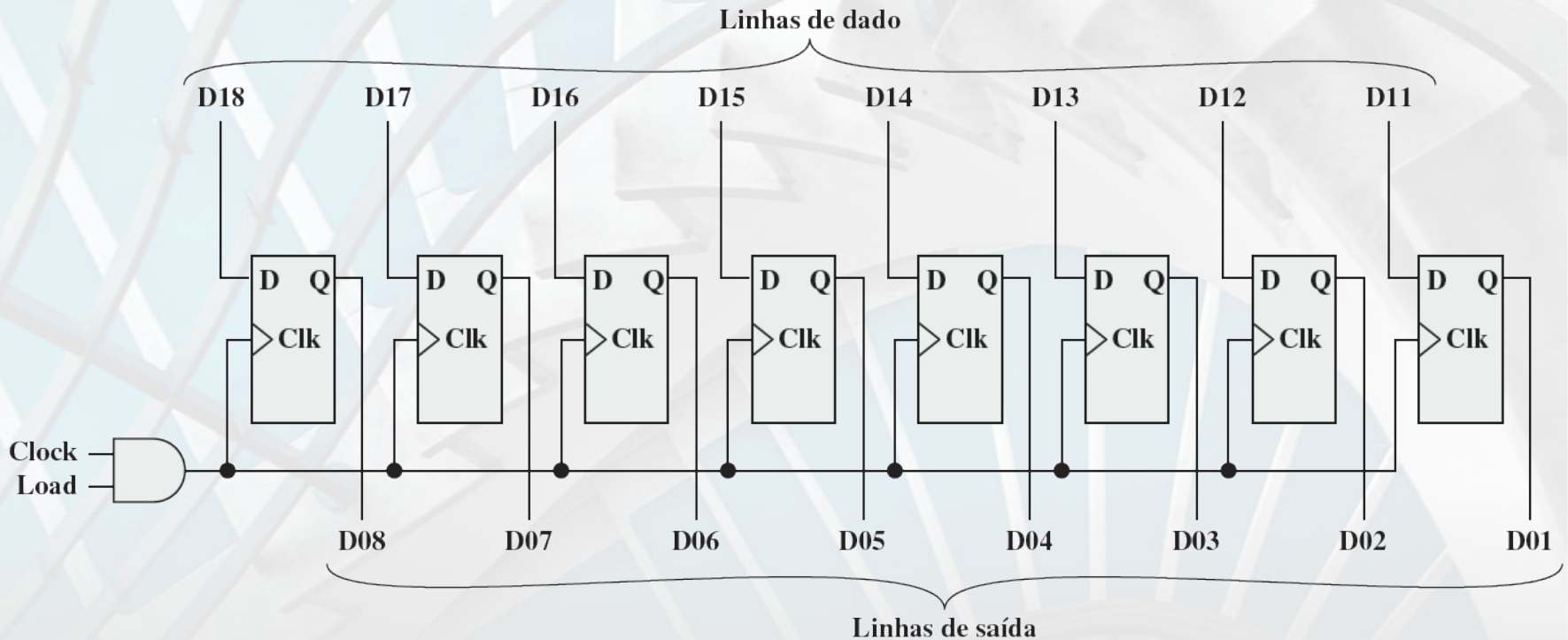
Nome	Símbolo gráfico	Tabela verdade															
S-R		<table><tr><th>S</th><th>R</th><th>Q_{n+1}</th></tr><tr><td>0</td><td>0</td><td>Q_n</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>–</td></tr></table>	S	R	Q_{n+1}	0	0	Q_n	0	1	0	1	0	1	1	1	–
S	R	Q_{n+1}															
0	0	Q_n															
0	1	0															
1	0	1															
1	1	–															
J-K		<table><tr><th>J</th><th>K</th><th>Q_{n+1}</th></tr><tr><td>0</td><td>0</td><td>Q_n</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>$\overline{Q_n}$</td></tr></table>	J	K	Q_{n+1}	0	0	Q_n	0	1	0	1	0	1	1	1	$\overline{Q_n}$
J	K	Q_{n+1}															
0	0	Q_n															
0	1	0															
1	0	1															
1	1	$\overline{Q_n}$															
D		<table><tr><th>D</th><th>Q_{n+1}</th></tr><tr><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td></tr></table>	D	Q_{n+1}	0	0	1	1									
D	Q_{n+1}																
0	0																
1	1																

Registradores

- Um **registrador** é um circuito digital usado dentro da CPU para armazenar um ou mais bits de dados.
- Dois tipos básicos de registradores costumam ser usados:
 1. Um **registrador paralelo** consiste em um conjunto de memória de 1 bit que pode ser lido ou escrito de modo simultâneo.
 2. Um **registrador de deslocamento** aceita e/ou transfere informação serialmente.

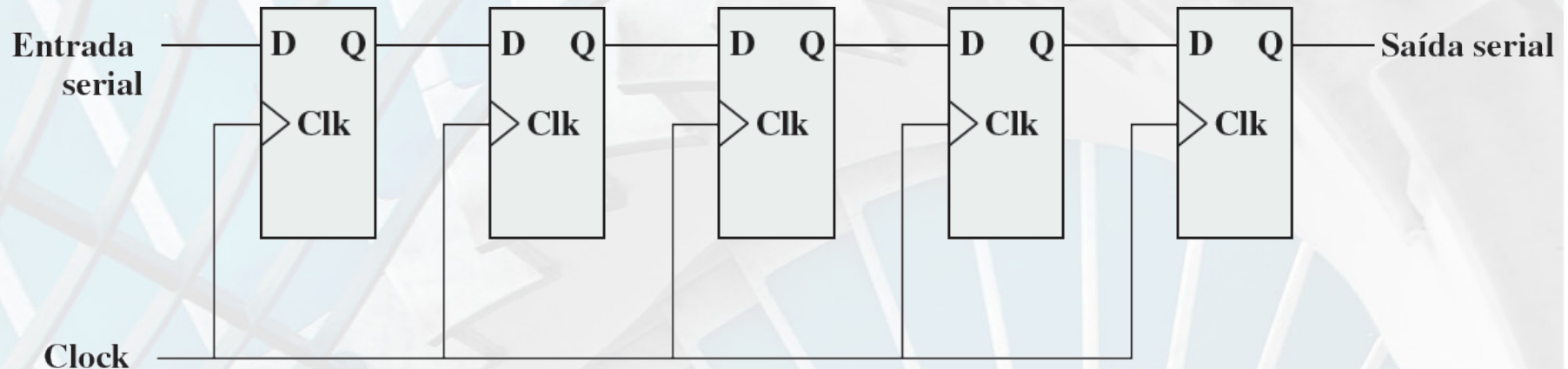
Registadores

- Registrador paralelo de 8 bits:



Registadores

- Registrador de deslocamento de 5 bits:

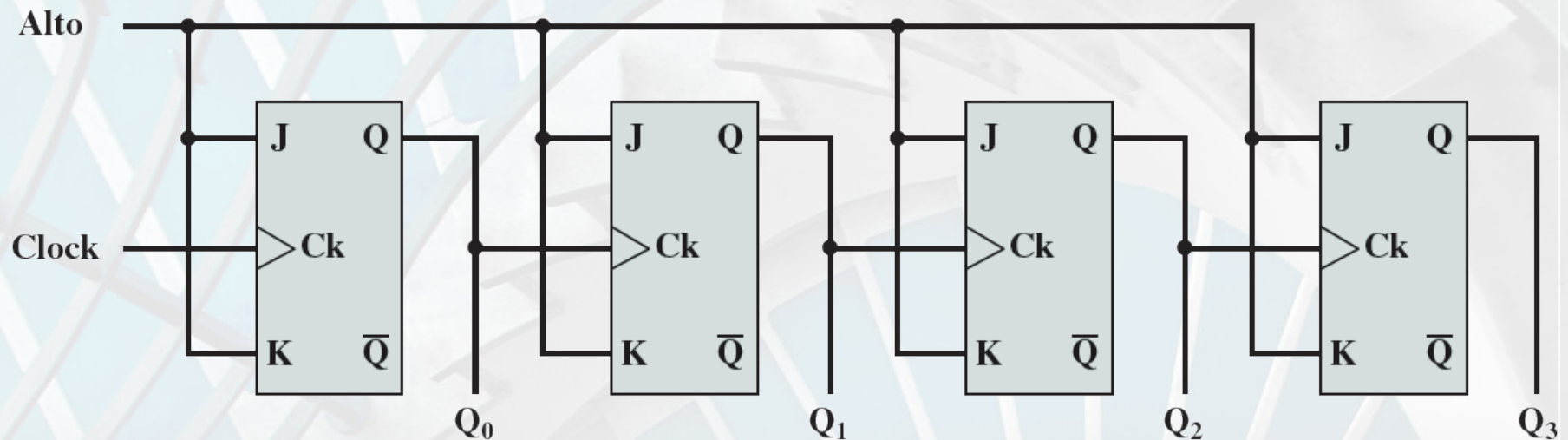


Contadores

- O **contador** é um registrador cujo valor é facilmente incrementado por 1 até atingir seu módulo, que é a capacidade do registrador.
- Os contadores podem ser designados como assíncronos ou síncronos, dependendo do modo como operam.
- Os **contadores assíncronos** são relativamente lentos porque a saída de um flip-flop dispara uma mudança no estado do flip-flop seguinte.
- Em um **contador síncrono**, todos os flip-flops mudam de estado ao mesmo tempo.

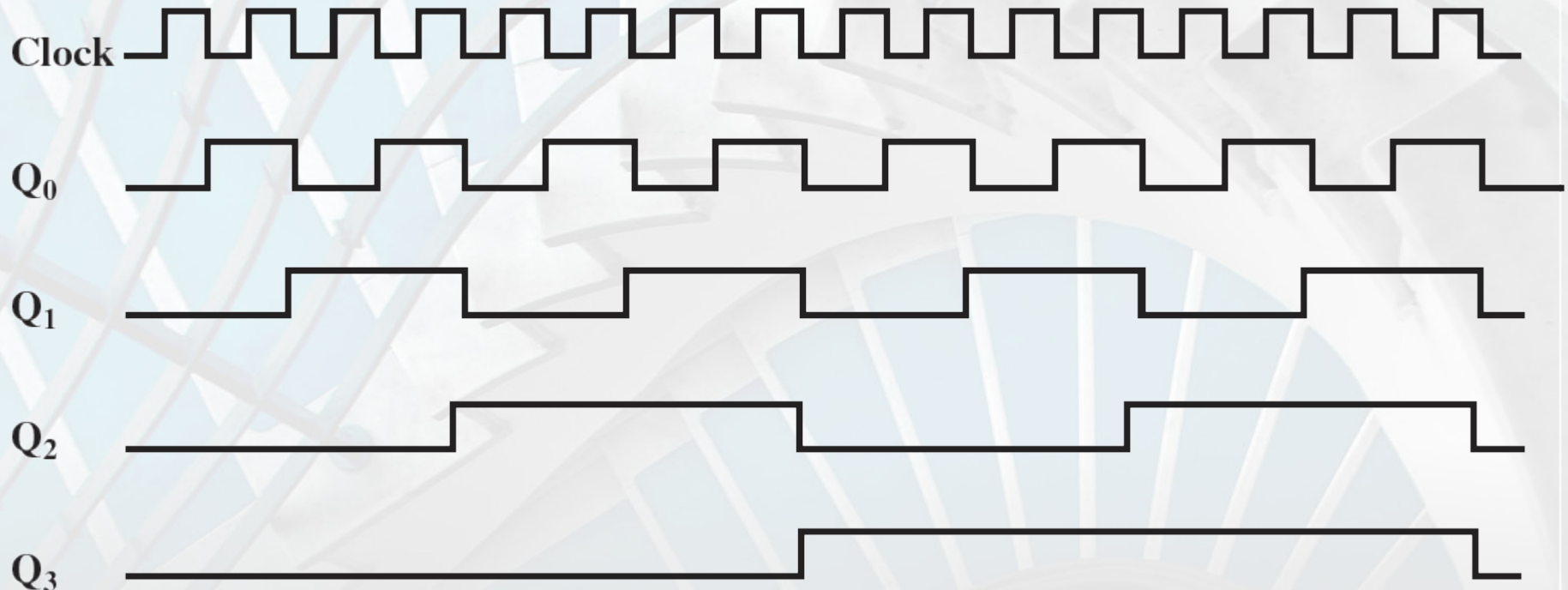
Contadores

- Contador de propagação - Circuito sequencial:



Contadores

- Contador de propagação - Diagrama de temporização:



Contadores

WILLIAM STALLINGS

Arquitetura e Organização de Computadores

10ª edição

- Modelo de um contador síncrono - Tabela verdade:

C	B	A	Jc	Kc	Jb	Kb	Ja	Ka
0	0	0	0	d	0	d	1	d
0	0	1	0	d	1	d	d	1
0	1	0	0	d	d	0	1	d
0	1	1	1	d	d	1	d	1
1	0	0	d	0	0	d	1	d
1	0	1	d	0	1	d	d	1
1	1	0	d	0	d	0	1	d
1	1	1	d	1	d	1	d	1

- Modelo de um contador síncrono - Mapas de Karnaugh:

$J_c = BA$

	BA			
	00	01	11	10
0			1	
1	d	d	d	d

$K_c = BA$

	BA			
	00	01	11	10
0	d	d	d	d
1			1	

$J_b = A$

	BA			
	00	01	11	10
0		1	d	d
1		1	d	d

$K_b = A$

	BA			
	00	01	11	10
0	d	d	1	
1	d	d	1	

$J_a = 1$

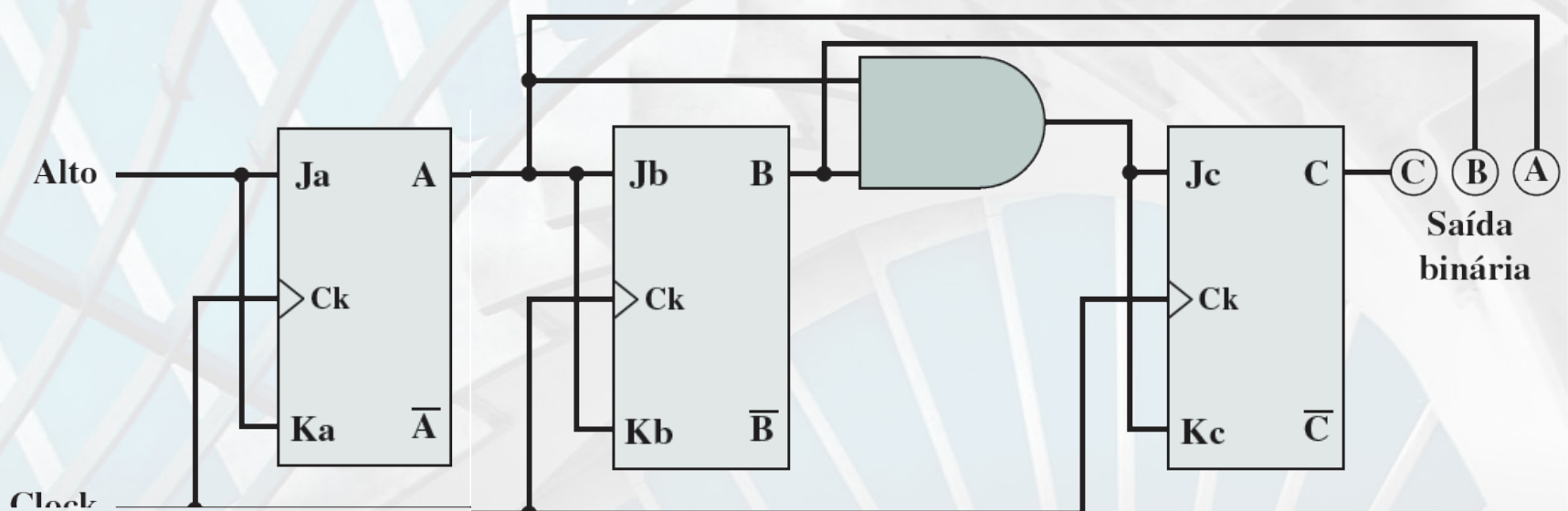
	BA			
	00	01	11	10
0	1	d	d	1
1	1	d	d	1

$K_a = 1$

	BA			
	00	01	11	10
0	d	1	1	d
1	d	1	1	d

Contadores

- Modelo de um contador síncrono - Diagrama lógico:



Dispositivos lógicos programáveis

WILLIAM STALLINGS

**Arquitetura e
Organização de
Computadores**

10ª edição

- **Dispositivo lógico programável (PLD)** – Um termo geral que se refere a qualquer tipo de circuito integrado usado para implementar hardware digital, em que o chip pode ser configurado pelo usuário final para realizar diferentes projetos.
- **Array lógico programável (PLA)** – Um PLD relativamente pequeno que contém dois níveis de lógica, um plano AND e um plano OR, em que ambos os níveis são programáveis.
- **Lógica de array programável (PAL)** – Um PLD relativamente pequeno que tem um plano AND programável seguido por um plano OR fixo.

Dispositivos lógicos programáveis

WILLIAM STALLINGS
**Arquitetura e
Organização de
Computadores**

10ª edição

- **PLD simples (SPLD)** – Um PLA ou PAL.
- **PLD complexo (CPLD)** – Um PLD mais complexo consiste em uma disposição de diversos blocos do tipo SPLD em um único chip.
- **Array de porta programável por campo (FPGA)** – FPGAs oferecem uma maior razão de flip-flops para recursos lógicos do que CPLDs.
- **Bloco lógico** – O termo bloco lógico é mais usado no contexto de FPGAs, mas podem também se referir a bloco de circuito em um CPLD.